

Оптимизированное матричное умножение

Владимир Попов

11 апреля 2026 г.

Содержание

1	Характеристики	3
2	Базовая версия	3
3	Лёгкие оптимизации	4
4	Тестирование	5
4.1	Корректность результата	5
4.2	Бенчмаркинг	6
5	Развёртка и перепланировка	9
6	Интринсики	11

1 Характеристики

Параметр	Значение
<i>Аппаратное обеспечение (Hardware)</i>	
Процессор (CPU)	AMD Ryzen 7 8845H (8 ядер, 16 потоков)
Архитектура	x86_64
Базовая / Максимальная частота	3.8 ГГц / 5.1 ГГц
L1 Кэш (данные/инструкции)	256 / 256 КиБ
L2 Кэш	8 МиБ
L3 Кэш	16 МиБ (общий)
Оперативная память	32 ГБ LPDDR5x
<i>Программное обеспечение (Software)</i>	
Операционная система	Ubuntu 22.04.5 LTS
Ядро Linux	6.8.0-106-generic
Компилятор	GCC 11.4 / Clang 11.4
Библиотека C	glibc 2.35
Флаги компиляции	-DNDEBUG -O3 -march=native -mprefer-vector-width=512 -ffast-math

Таблица 1: Характеристики тестовой системы и программного окружения

На моей машине есть поддержка следующих расширений:

AES, AMD-V AVX, AVX2, AVX512, FMA3, MMX-plus, SHA, SSE, SSE2, SSE3, SSE4.1, SSE4.2, SSE4A, SSSE3.

Также имеется 2 исполнительных FMA блока.

2 Базовая версия

Мы будем перемножать матрицы, содержащие элементы типа `fr32`. Писать будем на Си. Выделением памяти функция заниматься не должна, поэтому будем передавать указатель на итоговую матрицу в функцию. Будем считать, что на пользователя не возлагается ответственность занулять память, мало ли он использует `malloc`, поэтому должна будет сама занулять элементы.

Функция должна соответствовать следующему типу

```
1 typedef void (*matmul_t) (
```

```

2     const float* const mat1,
3     const float* const mat2,
4         float* const mat_dst,
5     const size_t i_size, const size_t j_size, const size_t k_size
6 );

```

, где `i_size` - количество строк в первой матрице, `j_size` - количество столбцов в первой матрице или же количество строк во второй матрице, `k_size` - количество столбцов во второй матрице.

Итоговая матрица будет размером $i_size \times k_size$.

Напишем базовую версию перемножения, чтобы сравнивать с ней корректность результата и ускорение.

```

1 void matmul_correct(
2     const float* const mat1,
3     const float* const mat2,
4         float* const mat_dst,
5     const size_t i_size, const size_t j_size, const size_t k_size
6 ) {
7     /* Asserts */
8     for (size_t i = 0; i < i_size; ++i) {
9         for (size_t k = 0; k < k_size; ++k) {
10             mat_dst[i * k_size + k] = 0;
11
12             for (size_t j = 0; j < j_size; ++j) {
13                 mat_dst[i * k_size + k] += mat1[i * j_size + j] * mat2[j * k_size + k];
14             }
15         }
16     }
17 }

```

3 Лёгкие оптимизации

Самая очевидная оптимизация, это изменение порядка вложенности циклов. В базовой версии мы идём по столбцам второй матрицы, а значит прыгаем по массиву с шагом равному `k_size`. Это соответствует плохой пространственной локальности, потому что на каждое новое обращение процессору требуется подгружать новую кеш-линию данных, хотя из старой обработан лишь один элемент.

Также логично потребовать гарантию от пользователя функции, что `mat1`, `mat2`, `mat_dst` не будут пересекаться, это всё таки разные матрицы. Для объявим указатели как `restrict`. Это позволит компилятору производить оптимизации, допустим не сохранять лишний раз результат, так как известно что он не перезапишется.

```
1 void matmul_optimized(  
2     const float* restrict const mat1,  
3     const float* restrict const mat2,  
4         float* restrict const mat_dst,  
5     const size_t i_size, const size_t j_size, const size_t k_size  
6 ) {  
7     /*Asserts*/  
8  
9     memset(mat_dst, 0, i_size * k_size * sizeof(*mat_dst));  
10  
11     for (size_t i = 0; i < i_size; ++i) {  
12         for (size_t j = 0; j < j_size; ++j) {  
13             for (size_t k = 0; k < k_size; ++k) {  
14                 mat_dst[i * k_size + k] += mat1[i * j_size + j] * mat2[j * k_size + k  
15             ];  
16         }  
17     }  
18 }
```

4 Тестирование

4.1 Корректность результата

Для проверки того, что оптимизированная версия корректно считает умножение матриц напишем функцию, проверяющую 2 матрицы на равенство. Значения будем сравнивать с точностью до какого-то знака, так как из-за отсутствия ассоциативности операций с типами с плавающей точкой может накапливаться погрешность.

```

1 bool mat_is_equal(
2     const float* const mat1,
3     const float* const mat2,
4     const size_t rows, const size_t cols
5 ) {
6     /* Asserts */
7     const size_t size = rows * cols;
8     for (size_t ind = 0; ind < size; ++ind) {
9         if (fabsf(mat1[ind] - mat2[ind]) > FLOAT_EPSILON) {
10             return false;
11         }
12     }
13
14     return true;
15 }

```

Я проверил корректность и при флаге `-march=native` получил погрешность результатов 2 знака после запятой. Изначально матрицы я заполнял случайными числами от 0 до 100, итоговые значения получались достаточно большими, поэтому относительная погрешность около 6 знаков, что в целом тоже много. Я подумал, что это из-за того, что в оптимизированной версии компилятор смог подставить `fma` инструкции, тем самым избежав потери точности, тогда как в неоптимизированной функции она осталась. Я скомпилировал с флагом `-ffp-contract=off`, который отключает `fma` инструкции и точность и вправду стала нормальной.

Я решил заполнять таблицу значения от 0 до 1, тогда итоговая погрешность около 5 знаков после запятой. Всё таки значения в оптимизированной функции считаются точнее, чем в обычной, поэтому имеет смысл при дальнейших оптимизациях сравнивать корректность именно с ней.

4.2 Бенчмаркинг

Для замера времени будем использовать обёртку над ассемблерной инструкцией `rdtsc`, возвращающую количество тактов. Нужно проводить много измерений, потом из них получать результат и ошибку. Я попробовал считать среднее значение, а также дисперсию, но дисперсия была огромной и очень сильно плавала от замера к замеру. Это можно связать с "шумом" от других приложений, изменением частоты и изменением контекста, если вдруг программа перебрасывается на другой процесс.

Я добавил при запуске фиксацию частоты на 3 ГГц, убрал AMD_BOOST, который тоже может сильно влиять на частоту, а также привязал программу к конкретному ядру и выставил большой приоритет, чтобы процесс не вытеснялся.

```
1 CPU_CORE = 0
2 GOVERNOR = performance
3 AMD_BOOST = /sys/devices/system/cpu/cpufreq/boost
4 FIXED_FREQ = 3000MHz
5
6 bench: clean_out
7 $(MAKE) DEBUG_=0 ADD_FLAGS="$(BENCH_FLAGS)" rebuild
8 @$(MAKE) setup_cpu
9 sudo taskset -c $(CPU_CORE) chrt -f 99 ./$(PROJECT_NAME).out $(OPTS)
10 @$(MAKE) restore_cpu
11
12 setup_cpu:
13 sudo cpupower frequency-set -u $(FIXED_FREQ) -d $(FIXED_FREQ) -g $(GOVERNOR)
14 if [ -f $(AMD_BOOST) ]; then echo 0 | sudo tee $(AMD_BOOST); fi
15
16 restore_cpu:
17 sudo cpupower frequency-set -g powersave
18 if [ -f $(AMD_BOOST) ]; then echo 1 | sudo tee $(AMD_BOOST); fi
```

Это не очень помогло. Шумы всё равно не перестали влиять. Логично было бы оценивать не среднее, а минимум, так как минимальное количество тактов показывает замер, в котором было наименьшее количество побочных факторов. А чтобы понять правда ли этот минимум является тем самым замером без побочных эффектов, нужно сравнить его с другими минимумами и посчитать дисперсию. Если она окажется маленькой, значит значение и вправду корректное.

Дисперсию будем считать через двухпроходный алгоритм, чтобы минимизировать ошибку связанную точностью чисел с плавающей точкой. А перед каждым замером будем прогревать кеш на тех же данных.

```
1 enum MatBenchError matbench(
2     const size_t bucket_iterations_cnt,
3     const size_t buckets_cnt,
4     const matmul_t matmul,
5     const size_t i_size, const size_t j_size, const size_t k_size,
6     double* mean_ptr, double* disp_ptr
7 ) {
8     /*Init something*/
```

```

9
10 double mean = 0;
11
12 for (size_t bucket = 0; bucket < buckets_cnt; ++bucket) {
13     double min = INFINITY;
14
15     for (size_t iteration = 0; iteration < bucket_iterations_cnt; ++iteration) {
16         matfill(mat1, i_size, j_size);
17         matfill(mat2, j_size, k_size);
18
19         matmul(mat1, mat2, mat_dst, i_size, j_size, k_size); // warming up
20
21         unsigned int $dummy$ = 0;
22         const unsigned long long start = __rdtscp(&$dummy$);
23
24         matmul(mat1, mat2, mat_dst, i_size, j_size, k_size);
25
26         const unsigned long long end = __rdtscp(&$dummy$);
27
28         const double time = (double)(end - start);
29         if (time < min) {
30             min = time;
31         }
32     }
33
34     buckets_mins[bucket] = min;
35     mean += min;
36 }
37
38 mean = mean / (double)buckets_cnt;
39
40 double disp = 0;
41
42 for (size_t time_ind = 0; time_ind < buckets_cnt; ++time_ind) {
43     const double x = buckets_mins[time_ind];
44     const double dev = x - mean;
45     disp += dev * dev;
46 }
47

```



```

48     disp = disp / (double)(buckets_cnt - 1);
49
50     /*Destroy something*/
51 }

```

Параметр бенчмарка	Значение
Размерность матриц ($I \times J \times K$)	$32 \times 64 \times 16$
Количество бакетов	10
Итераций в бакете	50000
Такты (matmul_correct)	88700 ± 170
Такты (matmul_optimized)	19500 ± 16
Ускорение	$4.55x \pm 0.01$

Таблица 2: Результаты бенчмаркинга производительности матричного умножения с наивными оптимизациями

Погрешность оказалась действительно маленькой, а значит скорее всего мы всё померили правильно и нашли реальный минимум.

5 Развёртка и перепланировка

Если сейчас посмотреть вывод при сборке программы с флагом `-fopt-info-vec-optimized`, то он покажет, что развёртка и использование `avx`-инструкций уже делается компилятором во внутреннем цикле по `k`.

```

1 # src/matrix/funcs.c:50:34 - for (size_t k = 0; k < k_size; ++k)
2 src/matrix/funcs.c:50:34: optimized: loop vectorized using 64 byte vectors
3 src/matrix/funcs.c:50:34: optimized: loop vectorized using 32 byte vectors
4 src/matrix/funcs.c:50:34: optimized: loop vectorized using 64 byte vectors
5 src/matrix/funcs.c:50:34: optimized: loop vectorized using 32 byte vectors

```

Но можно сделать эту развёртку более умной. Давайте класть в один `zmm`-регистр 16 значений `float` из строки второй матрицы, и дублировать соответствующее значение матрицы во второй регистр 16 раз. Тогда мы сможем за одну векторную `fma` операцию посчитать сразу 16 значений. Тогда аккумулярующим операндом этого `fma` также должен быть `zmm`-регистром, поэтому будем накапливать значения в дополнительных переменных, а после подсчёта за раз записывать их итоговую матрицу.

Так как `zmm`-регистров целых 32 штуки, а также на моей машине имеется 2 вычислительных блока `fma`, то можно также сделать развёртку по столбцам первой матрицы.

Хочется как можно позже уйти от кроссплатформенности функций, поэтому попробуем написать эту функцию без использования интринсиков напрямую. А также протестируем различные значения развёрток по столбцам.

Также стоит учесть, что для корректной работы некоторых интринсиков требуется, чтобы адреса были выровнены по 64 байта. Чтобы сообщить об этом компилятору будет использовать `__builtin_assume_aligned`, а выделять сами массивы теперь будем не через `calloc`, а через `aligned_alloc`.

```
1 #define BLOCK_HEIGHT_ (8)
2 #define BLOCK_WIDTH_ (16)
3 void matmul_optimized2(
4     const float* restrict const mat1,
5     const float* restrict const mat2,
6     float* restrict const mat_dst,
7     const size_t i_size, const size_t j_size, const size_t k_size
8 ) {
9     /*Assets*/
10
11     const float* __restrict a = __builtin_assume_aligned(mat1, ALIGNMENT);
12     const float* __restrict b = __builtin_assume_aligned(mat2, ALIGNMENT);
13     float* __restrict c = __builtin_assume_aligned(mat_dst, ALIGNMENT);
14
15     const size_t i_limit = i_size - (i_size % BLOCK_HEIGHT_);
16     const size_t k_limit = k_size - (k_size % BLOCK_WIDTH_);
17
18     for (size_t i = 0; i < i_limit; i += BLOCK_HEIGHT_) {
19         for (size_t k = 0; k < k_limit; k += BLOCK_WIDTH_) {
20
21             float acc0[BLOCK_WIDTH_] = {0};
22             /**/
23             float acc7[BLOCK_WIDTH_] = {0};
24
25             for (size_t j = 0; j < j_size; ++j) {
26
27                 const float* b_row = &b[j * k_size + k];
28
```

```

29     const float a0 = a[(i + 0) * j_size + j];
30     /**/
31     const float a7 = a[(i + 7) * j_size + j];
32
33     for (size_t v = 0; v < BLOCK_WIDTH_; ++v) {
34         acc0[v] += a0 * b_row[v];
35         /**/
36         acc7[v] += a7 * b_row[v];
37     }
38 }
39
40 for (size_t v = 0; v < BLOCK_WIDTH_; ++v) {
41     c[(i + 0) * k_size + k + v] = acc0[v];
42     /**/
43     c[(i + 7) * k_size + k + v] = acc7[v];
44 }
45 }
46 }
47
48 //Handle tails
49 }

```

Таблица 3: Результаты бенчмаркинга производительности матричного умножения с переупорядочиванием и развёрткой

Коэффициент развертки (i)	matmul_optimized Такты	matmul_optimized2 Такты	Ускорение
4	18840 ± 24	10070	$1.87x \pm 0.02$
8	18824 ± 18	3306	$5.72x \pm 0.02$
16	18820 ± 18	3760	$5x \pm 0.02$

6 Интринсики

Перепишем вторую оптимизированную версию с явным использованием интринсиков.

```

1 #define BLOCK_HEIGHT_ (8)
2 #define BLOCK_WIDTH_ (16)
3 void matmul_optimized3(

```

```

4  const float* restrict const mat1,
5  const float* restrict const mat2,
6      float* restrict const mat_dst,
7  const size_t i_size, const size_t j_size, const size_t k_size
8  ) {
9      /* Asserts */
10
11     const float* __restrict a = __builtin_assume_aligned(mat1, ALIGNMENT);
12     const float* __restrict b = __builtin_assume_aligned(mat2, ALIGNMENT);
13     float* __restrict c = __builtin_assume_aligned(mat_dst, ALIGNMENT);
14
15     const size_t i_limit = i_size - (i_size % BLOCK_HEIGHT_);
16     const size_t k_limit = k_size - (k_size % BLOCK_WIDTH_);
17
18     for (size_t i = 0; i < i_limit; i += BLOCK_HEIGHT_) {
19         for (size_t k = 0; k < k_limit; k += BLOCK_WIDTH_) {
20
21             __m512 acc0 = _mm512_setzero_ps();
22             /**/
23             __m512 acc7 = _mm512_setzero_ps();
24
25             for (size_t j = 0; j < j_size; ++j) {
26
27                 const __m512 b_row = _mm512_load_ps(b + (j * k_size + k));
28
29                 const __m512 a0 = _mm512_set1_ps(a[(i + 0) * j_size + j]);
30                 /**/
31                 const __m512 a7 = _mm512_set1_ps(a[(i + 7) * j_size + j]);
32
33                 acc0 = _mm512_fmadd_ps(a0, b_row, acc0);
34                 /**/
35                 acc7 = _mm512_fmadd_ps(a7, b_row, acc7);
36             }
37
38             _mm512_store_ps(c + ((i + 0) * k_size + k), acc0);
39             /**/
40             _mm512_store_ps(c + ((i + 7) * k_size + k), acc7);
41         }
42     }

```

```

43
44 // Tails
45 }

```

Параметр бенчмарка	Значение
Размерность матриц ($I \times J \times K$)	$32 \times 64 \times 16$
Количество бакетов	10
Итераций в бакете	50000
Такты (matmul_optimized2)	3306
Такты (matmul_optimized3)	2812
Ускорение	$1.17568x \pm 0.00001$

Таблица 4: Результаты бенчмаркинга производительности матричного умножения с интринсиками

Итоговое ускорение по сравнению с изначальной версией - **$29.0797x \pm 0.00427335$**